

SISTEMAS DE INFORMACIÓN PARA LA GESTIÓN

ICN-292 | Segundo semestre 2026 | Campus Vitacura

Laboratorio 3

Automatización de devoluciones con n8n

Nombre: José Castillo

Sección: 100

Profesor: José Gonzalez

Ayudantes: Catalina Bodaleo, Valeria Bobadilla y Benjamín Jara

Parámetros personales

S = 378 U = \$58.000 CLP D = 21 días

Repositorio GitHub

<https://github.com/jmcg583-byte/ICN292-Lab3-Castillo-Jose>

23 de septiembre de 2026

Universidad Técnica Federico Santa María

Resumen ejecutivo

Desarrollé un prototipo en n8n para automatizar la clasificación inicial de devoluciones de AndesHogar SpA. La solución recibe solicitudes por HTTP, aplica las reglas de negocio en orden, calcula el monto en UF y guarda la decisión. El objetivo es apoyar una primera respuesta más oportuna, frente al proceso manual descrito en el caso, que demora tres días hábiles y compromete una respuesta en 24 horas.

La implementación incluye tres workflows: triage, emisor y resumen. El triage tiene 11 nodos y utiliza Switch en modo Rules, sin nodo Code. Los parámetros personales son S = 378, U = \$58.000 y D = 21 días. El emisor prepara y envía las 15 solicitudes; el resumen agrupa las filas del registro mediante Summarize y construye un mensaje único.

Resultado del lote	Solicitudes	Monto CLP
APROBACION	4	\$125.950
REVISION	7	\$924.920
RECHAZO	4	\$200.950
DATOS_INVALIDOS	0	\$0
Total	15	\$1.251.820

La tasa de aprobación automática es $4/15 \times 100 = 26,7 \%$. Los totales coinciden con B-01-mensaje-resumen.png y con el cálculo sobre las solicitudes originales. Las pruebas de frontera aprueban un monto de \$58.000 y un plazo de 21 días. Las entradas con ID vacío y monto cero quedan en DATOS_INVALIDOS; el texto “veinte mil” hace fallar el Switch. El SKU inexistente ZZ-999 se aprueba porque el flujo todavía no consulta el catálogo.

Recomendación sobre el sistema definitivo

Recomiendo mantener este desarrollo como MVP y completar sus controles antes de utilizarlo como sistema definitivo. La clasificación del lote es consistente, pero las notificaciones solo se componen en nodos Set; el resumen aún no filtra por fecha y está inactivo en la exportación. También faltan validación del catálogo, manejo del día sin solicitudes, prevención de duplicados y alertas de error. El prototipo permite evaluar la lógica de decisión; estas pruebas no cuantifican todavía una reducción del tiempo de respuesta ni acreditan la entrega de mensajes a clientes.

Evidencia: tres workflows JSON, conjunto de 15 solicitudes y capturas identificadas por las partes A, B y C. Los anexos conservan sus nombres originales.

<https://github.com/jmcg583-byte/ICN292-Lab3-Castillo-Jose>

Índice

Resumen ejecutivo.....	1
1 Contexto y parámetros.....	3
2 Parte A Flujo de triage.....	3
2.1 Arquitectura de los workflows.....	3
2.2 Reglas de decisión y servicio UF.....	4
2.3 Resultados de las quince solicitudes.....	5
2.4 Emisor y URLs del webhook.....	6
3 Parte B Resumen programado.....	7
3.1 Configuración y resultado.....	7
3.2 Frecuencia y días sin solicitudes.....	7
4 Parte C Errores y casos borde.....	8
4.1 Tres entradas inválidas.....	8
4.2 Configuración On Error.....	9
4.3 Error Workflow y alertas.....	9
4.4 Límites de monto y plazo.....	10
4.5 Error de tipo intencional.....	10
4.6 Monto no numérico y SKU inexistente.....	11
5 Reproducción y conclusiones.....	12
Referencias y archivos de evidencia.....	13
Anexo A Evidencias del triage.....	14

Los anexos incluyen las capturas originales y la explicación de los datos visibles. La numeración C3, C4 y C5 de los archivos corresponde a las subdivisiones del enunciado.

1. Contexto y parámetros

AndesHogar SpA recibe solicitudes de devolución por un formulario web. El caso sitúa el cuello de botella en la decisión inicial de aprobar, rechazar o derivar a revisión. La automatización propuesta se concentra en ese triage y en la trazabilidad de su resultado; la inspección física del producto sigue siendo una tarea posterior.

La semilla personal se obtiene de los tres últimos dígitos del RUT, sin el dígito verificador. Para $S = 378$, los residuos son 28 al dividir por 50 y 2 al dividir por 4:

$$U = 30000 + 1000 \times (378 \bmod 50) = 58000 \text{ CLP}$$

$$D = 7 + 7 \times (378 \bmod 4) = 21 \text{ días}$$

Los archivos solicitudes.json y la hoja solicitudes de solicitudes.xlsx contienen los mismos 15 registros, con sus campos como texto. La hoja catalogo incorpora ocho SKU. La hoja parametros conserva la semilla de ejemplo 876; los workflows de este laboratorio utilizan explícitamente 378, \$58.000 y 21 días.

2. Parte A Flujo de triage

2.1. Arquitectura de los workflows

Workflow	Nodos	Función
triage	11	Recibir, clasificar, convertir a UF, registrar y responder.
emisor	4	Cargar las 15 solicitudes y enviar un POST por solicitud.
resumen	4	Consultar el registro, agrupar por ruta y componer un mensaje.

El triage comienza con Webhook y Consulta UF. Luego el Switch deriva a uno de cuatro Edit Fields: Datos inválidos, Rechazo, Revisión o Aprobación. Las cuatro ramas convergen en Monto en UF y continúan por Registro, Notificar cliente y Respond to Webhook. El diseño contiene 11 nodos y no incluye un nodo Code.

El emisor utiliza Manual Trigger, Cargar solicitudes, Split Out y Enviar solicitud. El resumen utiliza Schedule Trigger, Leer registro, Summarize y Mensaje resumen. Los nombres de los archivos exportados se mantienen como ICN292-Lab3-Castillo-Jose-triage.json, ICN292-Lab3-Castillo-Jose-emisor.json e ICN292-Lab3-Castillo-Jose-resumen.json.

2.2. Reglas de decisión y servicio UF

El Switch aplica la primera condición que se cumple. Send data to all matching outputs está desmarcado y el fallback adicional se llama APROBACION. Los campos del formulario se leen desde Webhook, porque el nodo anterior entrega la respuesta de la API. [1]

Orden	Condición	Salida
1	id_solicitud vacío	INVALIDO_CAMPO
2	monto.toNumber() ≤ 0	INVALIDO_MONTO
3	dias_desde_compra.toNumber() > 21	RECHAZO_PLAZO
4	estado_producto = danado_por_uso	RECHAZO_ESTADO
5	monto.toNumber() > 58000	REVISION_MONTO
6	estado_producto = con_fallas	REVISION_FALLA
Fallback	Ninguna de las condiciones anteriores	APROBACION

Las salidas 1 y 2 llegan a Datos inválidos; 3 y 4 a Rechazo; 5 y 6 a Revisión. Las comparaciones numéricas usan $>$ estricto. La igualdad con U o D no activa por sí sola una revisión o un rechazo; las demás condiciones continúan siendo aplicables.

Consulta UF ejecuta GET a <https://mindicador.cl/api/uf> y consume serie[0].valor. Monto en UF divide el monto de la solicitud por ese valor. Las capturas muestran \$40.999,93 por UF para el 23 de septiembre de 2026 y \$40.991,75 para el día anterior. Por ejemplo, $\$58.000 / \$40.999,93 = 1,41463652$ UF, consistente con la prueba DEV-L01.

Durante el desarrollo, una consulta HTTP ubicada después de la clasificación reemplazaba los datos del item y se perdían ruta y motivo. La solución utilizada fue mover Consulta UF antes del Switch y referenciar directamente Webhook y Consulta UF en las expresiones. Esta ubicación conserva el resultado de clasificación, pero hace que incluso una solicitud inválida dependa de la disponibilidad de la API.

Registro inserta en registro_devoluciones: id_solicitud, sku, monto, dias, estado_producto, ruta, motivo, U, D y monto_uf. Las salidas muestran además id, createdAt y updatedAt de la tabla. Cada Edit Fields conserva los datos y agrega ruta, motivo y parámetros.

Los motivos de rechazo y revisión agrupan dos causales en un solo texto. Aunque el orden del Switch es correcto, este texto no identifica por sí solo cuál de las dos reglas se activó. Registrar también la salida exacta del Switch mejoraría la auditoría.

2.3. Resultados de las quince solicitudes

La corrida del lote se realizó el 22 de septiembre por la URL de producción, según la bitácora de construcción. La tabla muestra el resultado de aplicar las reglas exportadas a las 15 entradas originales. Los totales coinciden con la captura B-01. Los códigos de motivo reproducen los textos definidos debajo de la tabla.

ID	Monto CLP	Días	Estado	Ruta	Motivo
DEV-1115	19.990	3	sin_abrir	APROBACION	M-A
DEV-1116	69.980	5	abierto_sin_uso	REVISION	M-V
DEV-1117	24.990	12	con_fallas	REVISION	M-V
DEV-1118	79.990	4	sin_abrir	REVISION	M-V
DEV-1119	45.990	25	danado_por_uso	RECHAZO	M-R
DEV-1120	89.990	2	con_fallas	REVISION	M-V
DEV-1121	289.990	9	sin_abrir	REVISION	M-V
DEV-1122	45.990	2	sin_abrir	APROBACION	M-A
DEV-1123	39.980	15	danado_por_uso	RECHAZO	M-R
DEV-1124	49.980	1	abierto_sin_uso	APROBACION	M-A
DEV-1125	34.990	22	sin_abrir	RECHAZO	M-R
DEV-1126	79.990	8	con_fallas	REVISION	M-V
DEV-C01	9.990	1	sin_abrir	APROBACION	M-A
DEV-C02	289.990	2	sin_abrir	REVISION	M-V
DEV-C03	79.990	29	abierto_sin_uso	RECHAZO	M-R

M-A: “Cumple todas las condiciones”.

M-V: “Monto sobre el umbral o producto con fallas”.

M-R: “Fuera de plazo o producto dañado por uso”.

Ruta	Cantidad	Monto total CLP
APROBACION	4	125.950
REVISION	7	924.920
RECHAZO	4	200.950
DATOS_INVALIDOS	0	0
Total	15	1.251.820

La tasa utiliza como denominador las 15 solicitudes del lote: $4/15 \times 100 = 26,7 \%$. DEV-C03 evidencia la precedencia: sus \$79.990 exceden U, pero sus 29 días exceden D y se clasifica en RECHAZO. DEV-1119 también se rechaza; la causal de plazo aparece antes que la de daño por uso. Estos resultados distinguen la regla activada del motivo combinado que guarda la implementación.

2.4. Emisor y URLs del webhook

Cargar solicitudes contiene un arreglo igual al archivo solicitudes.json. Split Out genera 15 items y Enviar solicitud realiza un POST por item; su cuerpo JSON se obtiene con `JSON.stringify($json)`. El endpoint espera POST: abrirlo en la barra del navegador enviaría GET y no reproduce la prueba.

La documentación oficial distingue la URL de prueba, registrada al escuchar un evento de prueba o ejecutar el workflow desde el editor, y la URL de producción, disponible al publicar el workflow. Las ejecuciones de producción se consultan en Executions. [2]

URL de prueba

<https://j0secastillo123.app.n8n.cloud/webhook-test/devoluciones-378>

URL de producción

<https://j0secastillo123.app.n8n.cloud/webhook/devoluciones-378>

Evidencia o conjunto	URL y alcance verificable
A-01 y A-02	La bitácora las asocia a pruebas unitarias realizadas con Test URL. Las imágenes muestran el recorrido de aprobación, sin la URL completa.
A-03	Fallo del emisor durante pruebas unitarias. La bitácora indica Test URL; el endpoint queda truncado en la captura.
Lote original de 15 solicitudes	Production URL, según las notas y la configuración final del emisor.
B-01	No utiliza webhook: corresponde al workflow de resumen.
C-INV, C3 y C5	Los paneles documentan los resultados de los nodos, pero no muestran la URL invocada. El emisor exportado apunta a producción; eso no acredita por sí solo la URL de cada ejecución histórica.
C4-01	El recorte contiene el error de tipo; no permite identificar el endpoint.

Notificar cliente prepara email_destino y mensaje en un nodo Set. El mensaje informa el identificador, la ruta y el motivo. No existe un nodo de correo o mensajería que lo entregue al destinatario; la notificación es una simulación. Respond to Webhook sí devuelve al emisor un JSON con id, ruta y motivo.

Ejemplo de respuesta que define el flujo para DEV-1122

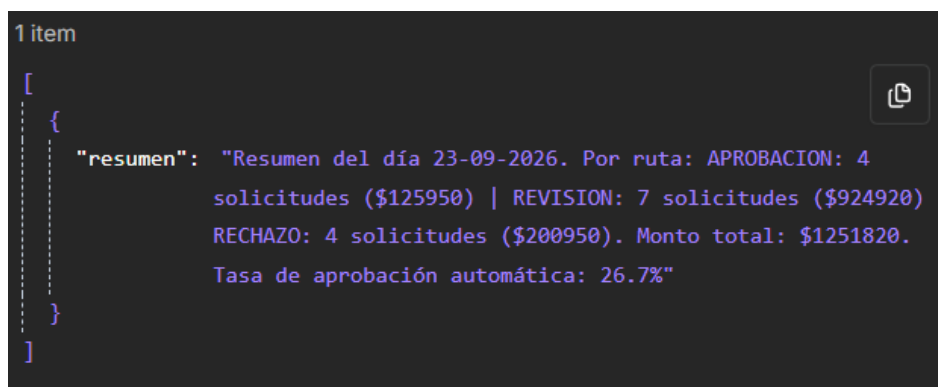
```
{"id": "DEV-1122", "ruta": "APROBACION",  
  "motivo": "Cumple todas las condiciones"}
```

3. Parte B Resumen programado

3.1. Configuración y resultado

Schedule Trigger está configurado para ejecutarse diariamente a las 20:00. Leer registro consulta registro_devoluciones; Summarize agrupa por ruta, cuenta id_solicitud y suma monto. Sus campos de salida son count_id_solicitud y sum_monto. Estas operaciones corresponden a Count y Sum del nodo de agregación. [3]

Mensaje resumen tiene Execute Once activado y utiliza \$input.all() para reunir los grupos en un único texto. La captura B-01 contiene un item y reporta 4 aprobaciones por \$125.950, 7 revisiones por \$924.920 y 4 rechazos por \$200.950. El total es \$1.251.820 y la tasa 26,7 %. DATOS_INVALIDOS no aparece en ese mensaje porque no tenía registros en el lote.



```
1 item
[
  {
    "resumen": "Resumen del día 23-09-2026. Por ruta: APROBACION: 4 solicitudes ($125950) | REVISION: 7 solicitudes ($924920) | RECHAZO: 4 solicitudes ($200950). Monto total: $1251820. Tasa de aprobación automática: 26.7%"
  }
]
```

Figura B1. B-01-mensaje-resumen.png. Salida del nodo Set con un único mensaje.

3.2. Frecuencia y días sin solicitudes

La frecuencia diaria a las 20:00 permite disponer de un corte de seguimiento al final de la jornada. El archivo exportado no declara timezone, por lo que utiliza la configuración de la instancia; no permite afirmar que las 20:00 correspondan a Chile. Además, active es false: la programación requiere publicar el workflow para operar automáticamente. [4]

La lectura no tiene un filtro por createdAt. Por eso el mensaje rotulado “Resumen del día 23-09-2026” representa las filas recuperadas del registro, aunque el lote original se procesó el 22. Las pruebas adicionales y los duplicados también pueden alterar una ejecución posterior. La etiqueta de fecha no convierte por sí sola la agregación en un resumen diario.

Un día sin solicitudes nuevas, si hay registros anteriores, el flujo volverá a resumirlos. Si la tabla no entrega items, no hay una rama explícita que produzca un mensaje de cero solicitudes. La tasa tampoco protege el denominador cero; estas son conclusiones de la configuración, no una prueba ejecutada del día vacío.

Para completar el comportamiento diario se debe fijar la zona horaria, filtrar una ventana temporal sin solapamientos, contemplar una salida “Sin solicitudes” con montos cero y tasa no aplicable, y conectar un nodo de envío. Si el corte se mantiene a las 20:00, conviene definir el día operativo entre dos cierres consecutivos. El nodo Set actual solo compone el texto.

4. Parte C Errores y casos borde

4.1. Tres entradas inválidas

Las tres pruebas son distintas: identificador vacío, monto cero y monto no numérico. Se muestran los campos de la carga respaldados por el registro visible o por la bitácora. Los campos no visibles en las capturas se omiten de estos fragmentos; no se presenta una reconstrucción como si fuera la carga original completa.

Caso 1 Identificador vacío

```
{"id_solicitud":"","sku":"PL-120", "monto":"25000",  
  "dias_desde_compra":"3", "estado_producto":"sin_abrir"}
```

Esperado: la primera regla deriva a DATOS_INVALIDOS. Observado: Registro guarda id_solicitud vacío, monto 25000, dias 3 y ruta DATOS_INVALIDOS, con U = 58000 y D = 21. Es una validación de negocio y el nodo termina correctamente. La captura acredita una cadena vacía; no demuestra una prueba con la propiedad completamente ausente.

Evidencia: C-INV-01-sin-id.png, figura C1.

Caso 2 Monto cero

```
{"id_solicitud":"DEV-Y01", "sku":"PL-120", "monto":"0",  
  "dias_desde_compra":"3", "estado_producto":"sin_abrir"}
```

Esperado: como el monto no es mayor que cero, corresponde DATOS_INVALIDOS. Observado: Registro guarda monto 0, monto_uf 0 y la ruta esperada. El ID válido evita que la primera regla determine el resultado. El nodo finaliza sin un error técnico.

Evidencia: C-INV-02-monto-cero.png, figura C2.

Caso 3 Monto escrito en palabras

```
{"id_solicitud":"DEV-X01", "monto":"veinte mil"}
```

Esperado: .toNumber() no puede interpretar ese texto como número y se produce un error. Observado: el Switch muestra “cannot convert to number”; el flujo no llega a Registro. Esta prueba cumple la condición de provocar una falla real de un nodo. El ID y el monto están consignados en la bitácora; la captura acredita el error, pero no muestra el cuerpo completo recibido por Webhook.

Evidencia: C-INV-03-veinte-mil-falla-nodo.png, figura C3; C5-01-veinte-mil-falla.png vuelve a documentar este comportamiento.

La ejecución C4 con DEV-C01 se analiza aparte: su entrada era válida y el error se provocó modificando la comparación del workflow. Por ello no se cuenta como una cuarta entrada inválida.

4.2. Configuración On Error

n8n ofrece tres comportamientos: Stop Workflow detiene la ejecución al fallar un nodo; Continue continúa por la salida habitual con los últimos datos válidos; Continue (using error output) entrega la información de error por una salida destinada a tratarlo. Esta última necesita una rama de manejo conectada para producir una respuesta útil. [5]

En esta implementación se mantuvo Stop Workflow, el valor predeterminado. Los JSON no contienen onError ni continueOnFail personalizados. La tabla agrupa nodos con el mismo criterio y cubre los tres workflows.

Nodos	Elección	Razón
Webhook, Manual Trigger y Schedule Trigger	Stop Workflow	Detener una ejecución cuyo inicio no se complete correctamente.
Cargar solicitudes y Split Out	Stop Workflow	Evitar envíos con una carga incompleta o mal estructurada.
Enviar solicitud	Stop Workflow	Hacer visible que un POST falló; no darlo por recibido.
Consulta UF	Stop Workflow	No calcular una conversión sin una respuesta válida.
Switch	Stop Workflow	Detener errores de tipo en vez de asignar una ruta arbitraria.
Datos inválidos, Rechazo, Revisión y Aprobación	Stop Workflow	Exigir ruta, motivo y parámetros antes de continuar.
Monto en UF y Registro	Stop Workflow	No informar un resultado si el cálculo o su persistencia fallan.
Notificar cliente y Respond to Webhook	Stop Workflow	Visibilizar una falla al componer el mensaje o la respuesta.
Leer registro, Summarize y Mensaje resumen	Stop Workflow	Evitar presentar como completo un resumen que falló.

4.3. Error Workflow y alertas

Un Error Workflow es un flujo de manejo de fallos que comienza con Error Trigger y se vincula desde Settings del workflow principal. Puede recibir el nombre del flujo, el nodo que falló y el detalle del error, y enviar una alerta. El Error Trigger se activa ante fallos de ejecuciones automáticas; la prueba manual del editor no lo dispara de la misma forma. [6, 7]

Los tres JSON no tienen un errorWorkflow asociado. Si falla el resumen programado, la ejecución queda con error, pero sin un aviso implementado el responsable puede no enterarse y el resumen no se genera. Esto no significa que el horario se desactive permanentemente: futuras ejecuciones pueden volver a intentarlo. Vincular una alerta permitiría detectar ese incumplimiento.

4.4. Límites de monto y plazo

Las dos solicitudes de frontera mantienen `id_solicitud`, SKU PL-120 y estado `sin_abrir`. Se modifica una variable a la vez y se deja la otra dentro del rango permitido.

Caso	Monto CLP	Días	Esperado	Observado
DEV-L01	58.000 = U	5	APROBACION	APROBACION
DEV-L02	30.000	21 = D	APROBACION	APROBACION

En DEV-L01, $58000 > 58000$ es falso; en DEV-L02, $21 > 21$ también es falso. Como las demás causales no se cumplen, ambas solicitudes llegan al fallback de aprobación. C3-01-limite-monto-igual-U.png y C3-02-limite-dias-igual-D.png muestran los campos registrados, el motivo y los parámetros aplicados; se incluyen como figuras C4 y C5.

4.5. Error de tipo intencional

La interfaz utilizada no ofrecía comparaciones de orden para String. La documentación también enumera mayor que y menor que dentro de Number, sin incluirlas en String. Por eso la prueba se adaptó: se retiró `.toNumber()` de la condición de monto y se mantuvo un operador Number con validación estricta. [1]

Se utilizó DEV-C01 del conjunto original, cuyo monto llega como el texto “9990”. El resultado fue una falla del Switch, no una ruta de revisión o aprobación. La captura C4-01-error-de-tipo.png contiene el mensaje que se muestra a continuación.

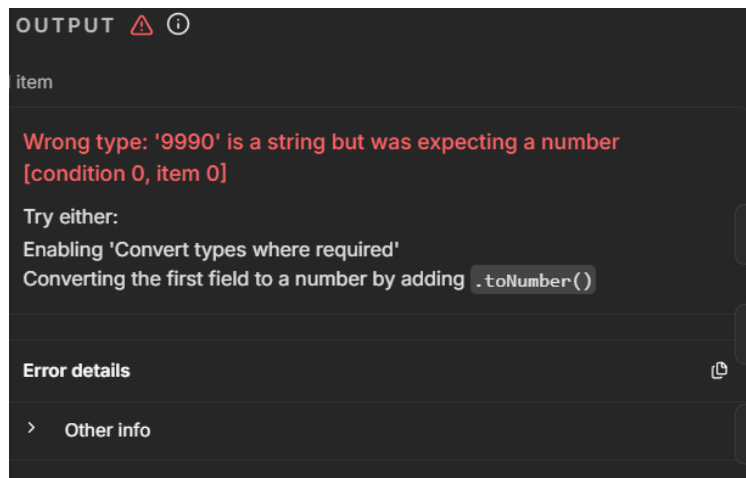


Figura C6. C4-01-error-de-tipo.png. Validación estricta del tipo de dato.

Explicación en dos líneas

El operador esperaba Number y recibió String; la validación estricta rechazó la comparación. Al restaurar `.toNumber()`, “9990” se convierte en 9990 y queda bajo U, si las demás reglas no se cumplen.

El JSON final vuelve a contener `.toNumber()` y el operador numérico > 58000 . Esta prueba acredita un error de tipos; no demuestra una comparación lexicográfica. La validación menos estricta intenta convertir según el operador, por lo que no corresponde afirmar que necesariamente compararía como texto. [1]

4.6. Monto no numérico y SKU inexistente

Caso monto “veinte mil”

La conversión `.toNumber()` no interpreta cantidades escritas con palabras.

El Switch falla antes del registro, con el mensaje “cannot convert to number”.

Se necesita validar el formato antes de convertir y responder por una ruta controlada de datos inválidos.

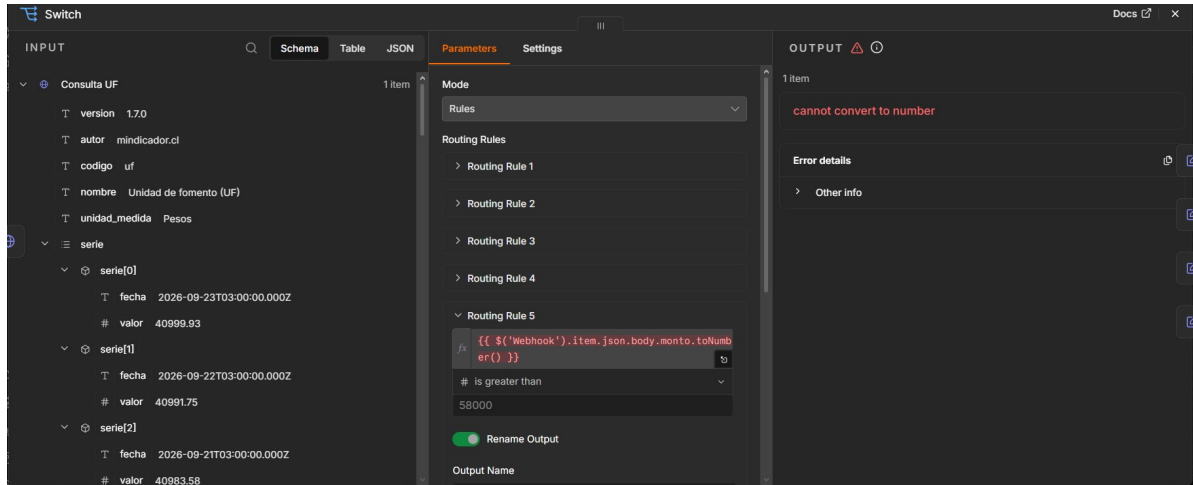


Figura C7. C5-01-veinte-mil-falla.png. Fallo visible del Switch al convertir el monto.

Caso SKU “ZZ-999”

Ninguna condición del Switch comprueba si el SKU pertenece al catálogo.

DEV-X02, por \$25.000 y 3 días, queda en APROBACION pese a usar ZZ-999, inexistente en catalogo.

Se requiere consultar el catálogo antes de decidir y tratar un SKU no encontrado como dato inválido.

C5-02-sku-inexistente-pasa.png muestra el registro de DEV-X02 con sku ZZ-999, ruta APROBACION y el motivo “Cumple todas las condiciones”. La figura C8 reproduce esa evidencia. El resultado es correcto respecto de las reglas programadas, pero esas reglas son insuficientes para garantizar que exista el producto.

La diferencia entre ambos casos orienta la mejora: el monto mal formado produce una interrupción visible, mientras que el SKU ajeno al catálogo produce una decisión aparentemente válida. La validación debe cubrir tanto el formato de cada campo como su coherencia con los datos del negocio.

5. Reproducción y conclusiones

Para reproducir el laboratorio se requiere una instancia n8n con los nodos utilizados, acceso a la API pública y una Data Table. No se necesitan credenciales para consultar mindicator.cl. Los JSON no incorporan las filas de registro_devoluciones; importar el workflow no reconstruye por sí solo esa tabla.

1. Importar los tres archivos JSON con sus nombres originales. Crear o seleccionar registro_devoluciones y reasignarla en Registro y Leer registro, porque su identificador depende de la instancia.
2. Crear columnas de texto para id_solicitud, sku, estado_producto, ruta y motivo; y columnas numéricas para monto, dias, U, D y monto_uf. La tabla agrega sus metadatos de fila.
3. Revisar $U = 58000$ y $D = 21$, las seis reglas y el fallback. Confirmar que pinData siga vacío. Las tres exportaciones entregadas tienen $\text{pinData} = \{\}$ y no contienen bloques de credenciales.
4. Publicar el triage y copiar su Production URL a Enviar solicitud. Para una prueba en el editor, usar Test URL mientras el Webhook está escuchando. Registrar la URL utilizada junto con la evidencia.
5. Ejecutar el emisor con una tabla destinada al lote original y revisar las 15 respuestas. Evitar mezclar las pruebas C con ese lote o repetirlo sin un control de duplicados.
6. Consultar los resultados del resumen. Para su operación diaria, completar el filtro temporal, la zona horaria, la salida sin solicitudes y el nodo de envío; después publicarlo y comprobar una ejecución programada.

El MVP demuestra que las reglas pueden implementarse de forma visible y reproducible, con rechazo antes que revisión y conversiones numéricas explícitas. Las pruebas de frontera son consistentes con la política. El lote original permite identificar que 4 de 15 solicitudes podrían resolverse por aprobación automática; no constituye una estimación del comportamiento de todas las devoluciones futuras.

Para una siguiente versión priorizaría el envío real de mensajes y el resumen diario, la validación de tipos y catálogo, motivos específicos por causal, el control de duplicados y las alertas operativas. También aislaría la conversión a UF para que una caída del servicio económico no bloquee la validación básica de una devolución. Estas mejoras sustentan la recomendación de mantenerlo como MVP antes de una operación definitiva.

La captura A-03 muestra una falla del emisor durante el desarrollo. La bitácora describe un problema de JSON en Respond to Webhook y una ejecución posterior exitosa; el recorte A-03 no muestra el detalle del mensaje de error. Las figuras de C aportan evidencia más precisa de fallos y de resultados registrados.

Referencias y archivos de evidencia

Documentación oficial consultada el 23 de septiembre de 2026. Las referencias se utilizan para explicar el comportamiento de n8n; los resultados del laboratorio provienen de los archivos, capturas y bitácora de construcción.

[1] **n8n. Switch.**

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.switch>

[2] **n8n. Webhook.**

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook>

[3] **n8n. Summarize.**

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.summarize>

[4] **n8n. Schedule Trigger.**

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.scheduletrigger>

[5] **n8n. Work with nodes.**

<https://docs.n8n.io/build/understand-workflows/workflow-components/work-with-nodes>

[6] **n8n. Error Trigger.**

<https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.errortrigger>

[7] **n8n. Handle errors gracefully.**

<https://docs.n8n.io/build/flow-logic/handle-errors-gracefully>

Datos y material de trabajo

Enunciado del Laboratorio 3 ICN-292; solicitudes.json; solicitudes.xlsx; notas-lab3.md; los tres workflows exportados y las capturas A, B y C. La API utilizada es <https://mindicador.cl/api/uf>. Las pruebas adicionales se mantienen separadas de los totales del conjunto original en este informe.

Correspondencia de evidencias

Archivos originales	Ubicación
A-01, A-02 y A-03	Anexo A, figuras A1 a A3
B-01-mensaje-resumen.png	Parte B, figura B1
C-INV-01, C-INV-02 y C-INV-03	Anexos de entradas inválidas, figuras C1 a C3
C3-01 y C3-02	Anexos de casos límite, figuras C4 y C5
C4-01-error-de-tipo.png	Parte C, figura C6
C5-01-veinte-mil-falla.png	Parte C, figura C7
C5-02-sku-inexistente-pasa.png	Anexo de catálogo, figura C8

<https://github.com/jmcg583-byte/ICN292-Lab3-Castillo-Jose>

Anexo A Evidencias del triage

A-01 y A-02 muestran el recorrido de un ítem por la salida de aprobación. A-03 documenta una ejecución fallida del emisor. Estos recortes se complementan con los paneles de Registro y Switch de los anexos siguientes.

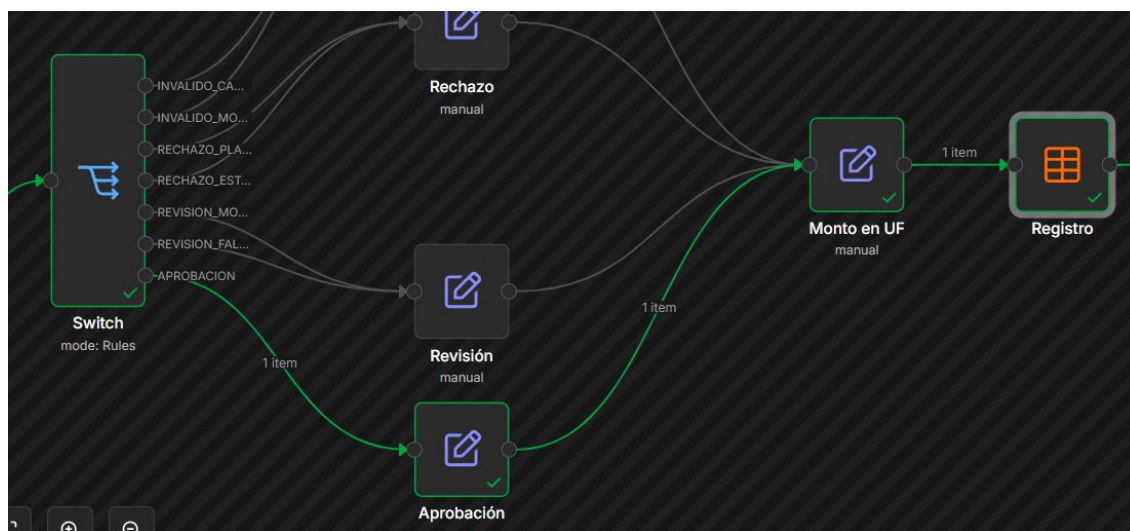


Figura A1. A-01-exito-flujo-completo.png. Recorrido visible por Aprobación, Monto en UF y Registro.

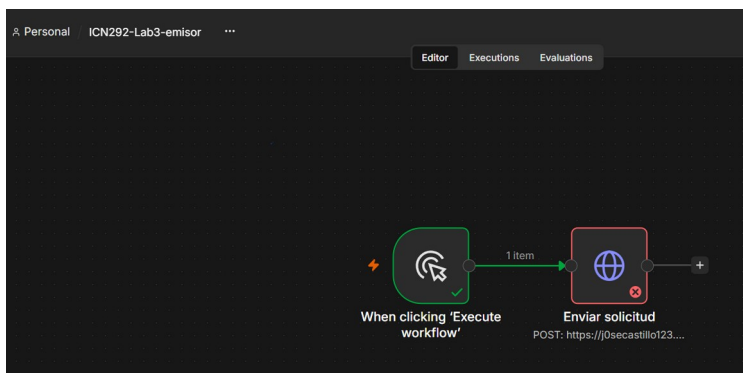
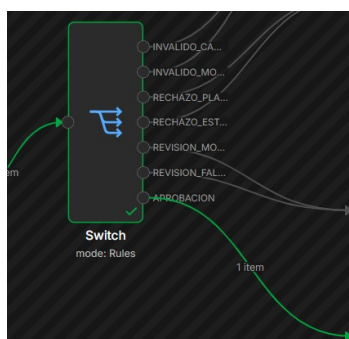


Figura A2, izquierda. A-02-exito-switch-aprobacion.png. Salida APROBACION.

Figura A3, derecha. A-03-ejecucion-fallida.png. Nodo Enviar solicitud con error.

La figura A1 conserva el nombre original del archivo, aunque la imagen encuadra solo parte del workflow. La figura A3 no expone los paneles de entrada y salida; las figuras C1 a C5 y C8 permiten examinar los datos registrados y el fallo del Switch con mayor detalle.

Anexo C1 Identificador vacío

Entrada relevante: id_solicitud vacío, monto 25000, 3 días y estado sin_abrir. La primera regla debe detener la evaluación de negocio y asignar DATOS_INVALIDOS.

The screenshot displays the 'Registro' application interface, which is divided into three main sections: INPUT, Parameters, and OUTPUT.

INPUT: This section shows a list of input data items. The first item is 'Monto en UF' with a value of 1.70. Below it, there are several other items including 'autor', 'codigo', 'nombre', 'unidad_medida', and a series of items labeled 'serie[0]', 'serie[1]', and 'serie[2]'. Each series item contains 'fecha' and 'valor' fields.

Parameters: This section contains configuration options for the data table. The 'Resource' is set to 'Row', the 'Operation' is 'Insert', and the 'Data table' is 'registro_devoluciones'. The 'Mapping Column Mode' is set to 'Map Each Column Manually'. Under 'Values to insert', the 'id_solicitud' field is mapped to the expression `{{ $('Webhook').item.json.body.id_solicitud }}` and the 'sku' field is mapped to `{{ $('Webhook').item.json.body.sku }}`.

OUTPUT: This section shows the resulting JSON output for the first item. The JSON object contains the following fields: 'id_solicitud' (empty string), 'sku' ('PL-120'), 'monto' (25000), 'dias' (3), 'estado_producto' ('sin_abrir'), 'ruta' ('DATOS_INVALIDOS'), 'motivo' ('Falta id_solicitud o monto no es mayor que cero'), 'U' (58000), 'D' (21), 'monto_uf' (0.6097571386097489), 'id' (23), 'createdAt' ('2026-09-23T21:46:33.198Z'), and 'updatedAt' ('2026-09-23T21:46:33.198Z').

Figura C1. C-INV-01-sin-id.png.

Resultado visible: Registro guarda DATOS_INVALIDOS, U = 58000 y D = 21. El monto convertido es 0,60975714 UF. El nodo termina correctamente; no se trata de un fallo técnico. La fila muestra createdAt = 2026-09-23T21:46:33.198Z.

Anexo C2 Monto cero

Entrada relevante: DEV-Y01, monto 0, 3 días, SKU PL-120 y estado sin_abrir. La segunda regla debe asignar DATOS_INVALIDOS porque el monto no es mayor que cero.

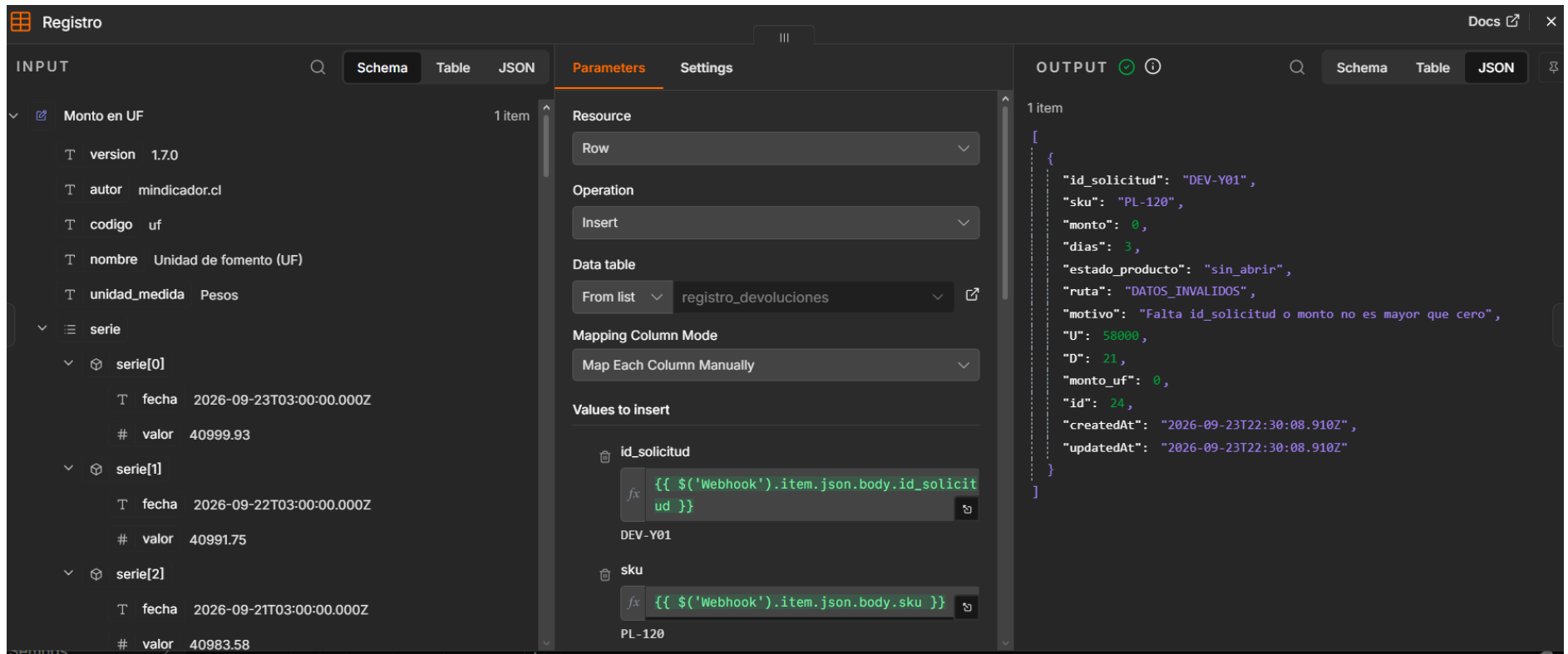


Figura C2. C-INV-02-monto-cero.png.

Resultado visible: la ruta es DATOS_INVALIDOS y monto_uf = 0. Se conserva el identificador de la solicitud y los parámetros personales. La fila muestra createdAt = 2026-09-23T22:30:08.910Z.

Anexo C3 Falla de conversión

Prueba identificada en la bitácora como DEV-X01 con monto “veinte mil”. La captura muestra el nodo Switch, la referencia al monto mediante `.toNumber()` y el panel de error.

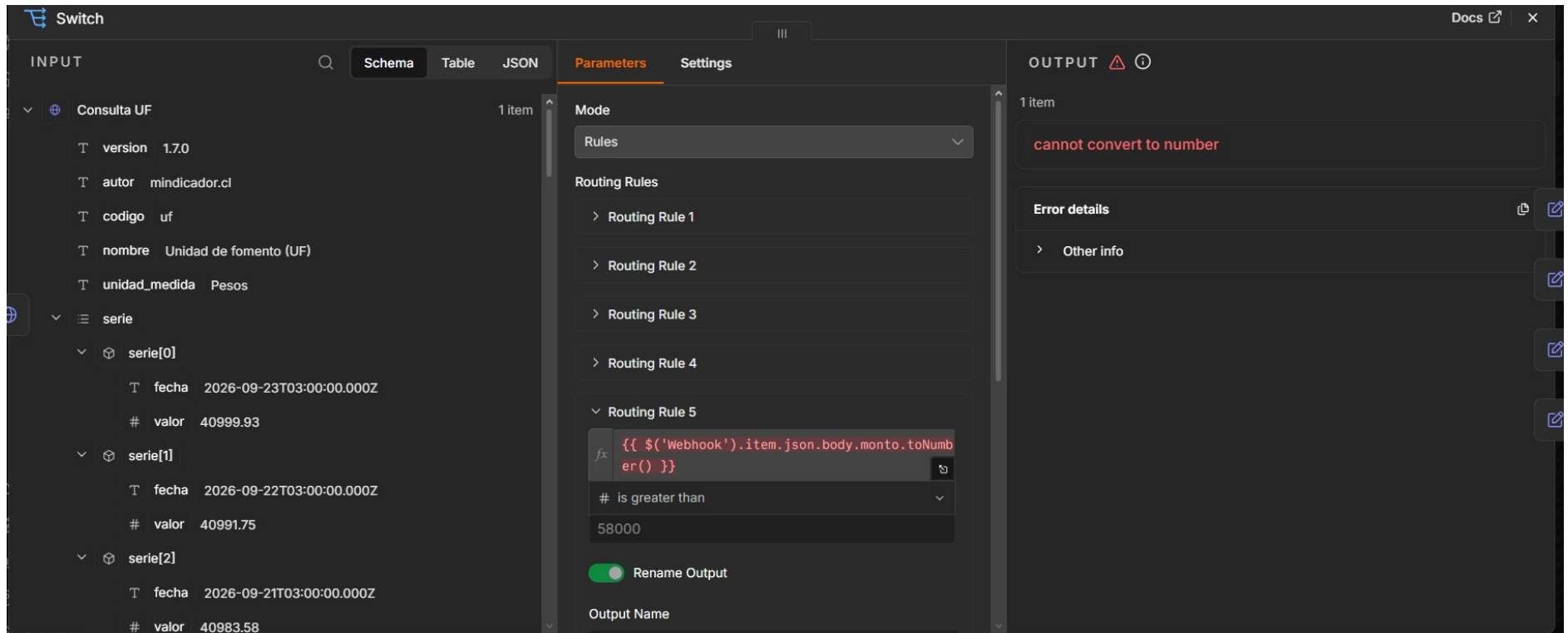


Figura C3. C-INV-03-veinte-mil-falla-nodo.png.

Resultado visible: “cannot convert to number”. El error impide continuar hasta Registro. El panel de entrada corresponde a Consulta UF; el cuerpo original del Webhook no está desplegado en esta imagen.

Anexo C4 Monto igual al umbral

Entrada relevante: DEV-L01, monto 58000 = U, 5 días, SKU PL-120 y estado sin_abrir. La condición monto > U debe resultar falsa.

The screenshot displays a workflow configuration interface with three main panels: INPUT, Parameters, and OUTPUT.

INPUT Panel: Shows a single item with the following structure:

- Monto en UF:** 1 item
 - version: 1.7.0
 - autor: mindicador.cl
 - codigo: uf
 - nombre: Unidad de fomento (UF)
 - unidad_medida: Pesos
- serie:**
 - serie[0]:
 - fecha: 2026-09-23T03:00:00.000Z
 - valor: 40999.93
 - serie[1]:
 - fecha: 2026-09-22T03:00:00.000Z
 - valor: 40991.75
 - serie[2]:
 - fecha: 2026-09-21T03:00:00.000Z
 - valor: 40983.58

Parameters Panel: Shows configuration for a resource and operation.

- Resource:** Row
- Operation:** Insert
- Data table:** From list (registro_devoluciones)
- Mapping Column Mode:** Map Each Column Manually
- Values to insert:**
 - id_solicitud:** `{{ $('Webhook').item.json.body.id_solicitud }}` (DEV-L01)
 - sku:** `{{ $('Webhook').item.json.body.sku }}` (PL-120)

OUTPUT Panel: Shows the resulting JSON output for 1 item:

```
[
  {
    "id_solicitud": "DEV-L01",
    "sku": "PL-120",
    "monto": 58000,
    "dias": 5,
    "estado_producto": "sin_abrir",
    "ruta": "APROBACION",
    "motivo": "Cumple todas las condiciones",
    "U": 58000,
    "D": 21,
    "monto_uf": 1.4146365615746173,
    "id": 18,
    "createdAt": "2026-09-23T19:55:28.379Z",
    "updatedAt": "2026-09-23T19:55:28.379Z"
  }
]
```

Figura C4. C3-01-limite-monto-igual-U.png.

Resultado visible: APROBACION, motivo “Cumple todas las condiciones”, U = 58000 y D = 21. El registro muestra monto_uf = 1,41463652 aproximadamente y createdAt = 2026-09-23T19:55:28.379Z.

Anexo C5 Días iguales al plazo

Entrada relevante: DEV-L02, monto 30000, 21 días = D, SKU PL-120 y estado sin_abrir. La condición $\text{dias_desde_compra} > D$ debe resultar falsa.

The screenshot displays the 'Registro' (Register) interface in a dark theme. It is divided into three main sections: INPUT, Parameters, and OUTPUT.

- INPUT:** Shows a list of fields for a record. The 'serie' field is expanded, showing three items (serie[0], serie[1], serie[2]) with their respective 'fecha' and 'valor'.
- Parameters:** Contains configuration for the 'Resource' (Row), 'Operation' (Insert), 'Data table' (registro_devoluciones), and 'Mapping Column Mode' (Map Each Column Manually). Under 'Values to insert', the 'id_solicitud' and 'sku' fields are mapped using JavaScript expressions: `{{ $('Webhook').item.json.body.id_solicitud }}` and `{{ $('Webhook').item.json.body.sku }}`.
- OUTPUT:** Displays the resulting JSON object for one item, containing fields like 'id_solicitud', 'sku', 'monto', 'dias', 'estado_producto', 'ruta', 'motivo', 'U', 'D', 'monto_uf', 'id', 'createdAt', and 'updatedAt'.

Figura C5. C3-02-limite-dias-igual-D.png.

Resultado visible: APROBACION, motivo “Cumple todas las condiciones”, U = 58000 y D = 21. El registro muestra $\text{monto_uf} = 0,73170857$ aproximadamente y $\text{createdAt} = 2026-09-23T19:55:28.665Z$.

Anexo C8 SKU inexistente

Entrada relevante: DEV-X02, SKU ZZ-999, monto 25000, 3 días y estado sin_abrir. El catálogo entregado en solicitudes.xlsx no contiene ZZ-999.

The screenshot displays the configuration of a 'Registro' node in a data pipeline tool. The interface is divided into three main sections: INPUT, Parameters, and OUTPUT.

INPUT: Shows a table named 'Monto en UF' with 1 item. The table structure includes columns for version (1.7.0), autor (mindicador.cl), codigo (uf), nombre (Unidad de fomento (UF)), and unidad_medida (Pesos). It also includes a 'serie' array with three items, each containing a 'fecha' and a 'valor'.

Parameters: This section is used to configure the data source and operation. It includes fields for 'Resource' (set to 'Row'), 'Operation' (set to 'Insert'), 'Data table' (set to 'registro_devoluciones'), 'Mapping Column Mode' (set to 'Map Each Column Manually'), and 'Values to insert'.

Values to insert: This section shows the mapping of values to the 'id_solicitud' and 'sku' columns. The 'id_solicitud' is mapped to 'DEV-X02' and the 'sku' is mapped to 'ZZ-999'.

OUTPUT: Shows the resulting JSON output for the 'Registro' node. The output is a single item with the following fields: 'id_solicitud' (DEV-X02), 'sku' (ZZ-999), 'monto' (25000), 'dias' (3), 'estado_producto' (sin_abrir), 'ruta' (APROBACION), 'motivo' (Cumple todas las condiciones), 'U' (58000), 'D' (21), 'monto_uf' (0.6097571386097489), 'id' (22), 'createdAt' (2026-09-23T20:51:22.066Z), and 'updatedAt' (2026-09-23T20:51:22.066Z).

Figura C8. C5-02-sku-inexistente-pasa.png.

Resultado visible: el nodo Registro acepta el SKU y guarda APROBACION. Esto evidencia que la validación no consulta el catálogo. La fila muestra createdAt = 2026-09-23T20:51:22.066Z.